

Consegna 7b-CUDA

Fattorizzazione RSA su GPU

Limiti Architettureali e Implementazioni di Base

L'implementazione CUDA della fattorizzazione RSA presenta limiti architetturali rispetto alle versioni per CPU:

- **Tipi di dato:** l'assenza della libreria `BIGNUM` impone l'uso di interi `unsigned long long` nativi, limitando lo spazio di ricerca a moduli a 64 bit.
- **Early Termination:** l'interruzione asincrona è inefficiente. Un'istruzione `return` isolata genera *Warp Divergence*. L'uso di variabili globali (`__managed__ int found`) costringe migliaia di thread a eseguire un *polling* continuo sulla global memory, saturando il bus PCIe.

Il codice originale implementa il **Grid-Stride Loop**, spazzolando il dominio tramite salti pari allo *stride* (`gridDim.x * blockDim.x`).

```
[mariaandrea.zoccoli@ui01 factorize]$ cat 14956824208499520709.txt
m=9223372036854775807  grid_dim=2, block_dim=32  stride=64 max_check=3037000499
Primo fattore trovato: 16513
19.995
```

OSS: il tempo elevato (~20s) è dovuto al forte sotto-utilizzo dell'hardware (*low occupancy*): lanciare soli 64 thread su una Tesla P100 lascia la GPU quasi interamente in stato di *idle*.

Ottimizzazioni Implementate

Per abbattere i tempi, la GPU è stata saturata (1024 blocchi x 1024 thread) e sono state testate due strategie.

Uso della Shared Memory

È stato allocato un flag locale `__shared__ int block_found` nella cache L1 di ogni multiprocessore. Non appena un thread trova il fattore, alza il flag condiviso fermando immediatamente tutti i thread del suo blocco senza interrogare la global memory.

```
m=14956824208499520709  grid_dim=1024, block_dim=1024  stride=1048576
Primo fattore trovato: 3586241843
0.161
```

Decomposizione a Chunk Contigui

Sostituzione del pattern grid-stride con l'assegnazione a ogni thread di un intervallo contiguo di numeri da testare.

```
m=14956824208499520709  grid_dim=1024, block_dim=1024  stride=1048576
Primo fattore trovato: 3586241843
0.199
```

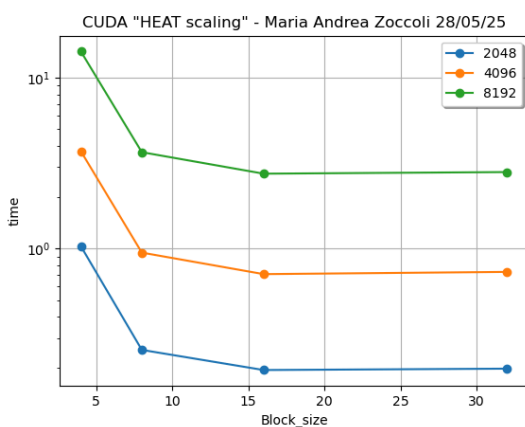
OSS: il tempo della memoria Shared risulta il migliore assoluto (0.16s). La decomposizione a chunk (0.19s) soffre di *load imbalance*: essendo la ricerca non deterministica, i thread con intervalli numerici alti eseguono calcoli inutili fino alla propagazione del segnale globale di fine.

Simulazione Propagazione del Calore

Flusso di Base

Il codice alloca tre buffer *application lifetime* (`D_T_const`, `D_T_old`, `D_T`). Il ciclo iterativo sulla CPU prevede il lancio sequenziale dei kernel `copy_constant` e `Jacobi_Iterator_GPU`, separati da barriere `cudaDeviceSynchronize()`, terminando con lo *swap* dei puntatori (operazione a costo zero). Trattandosi di un problema *Memory-Bound*, le letture/scritture multiple sulla Global Memory costituiscono il principale collo di bottiglia.

Analisi dello Scaling



Estratto dei tempi base

```
CUDA, 2048, 2048, 4, 4 ... 1.031s
CUDA, 2048, 2048, 16, 16 ... 0.194s
CUDA, 4096, 4096, 16, 16 ... 0.709s
CUDA, 8192, 8192, 16, 16 ... 2.746s
```

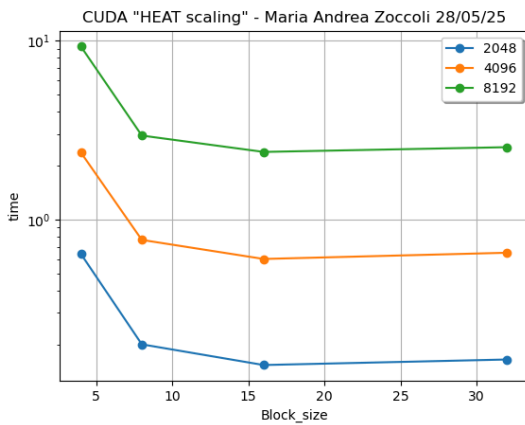
- **4x4:** prestazioni pessime, vengono lanciati solo 16 thread fisici. Poiché lo scheduler hardware raggruppa sempre i thread in warp da 32, questa configurazione lascia i core inattivi per il 50% del tempo, inoltre sovraccarica l'host con milioni di blocchi minuscoli.
- **16x16:** è la dimensione aurea. 256 thread per blocco garantiscono un'*occupancy* ideale dei Multiprocessori. Questa configurazione satura i core e permette all'hardware di mascherare le latenze di memoria eseguendo altri Warp mentre si attendono i dati in Global.
- **Scalabilità sui Dati:** Quadruplicando l'area della griglia (da 2048 a 4096), il tempo di risoluzione quadruplica in modo quasi esatto (da ~0.19s a ~0.70s). Questo comportamento dimostra un perfetto bilanciamento del carico limitato esclusivamente dalla saturazione lineare della banda passante.

Ottimizzazione: Single Kernel e Shared Memory

Il codice è stato riscritto per limitare l'accesso al bus di memoria:

1. **Fusione Kernel:** la logica di ripristino dei bordi costanti è stata integrata dentro il calcolo di Jacobi, eliminando un ciclo intero di I/O.
2. **Memoria Shared:** la griglia è stata suddivisa in "tile" caricati nella cache L1, incluse le ghost celss, azzerando i cache miss causati dallo stencil a 4 punti.

Analisi dello scaling (`cuda_heat_shared_single.cu`)



Tempi Ottimizzati (Shared Single Kernel)

```
CUDA, 2048, 2048, 4, 4 ... 0.640
CUDA, 2048, 2048, 16, 16 ... 0.153
CUDA, 4096, 4096, 16, 16 ... 0.600
CUDA, 8192, 8192, 16, 16 ... 2.382
CUDA, 8192, 8192, 32, 32 ... 2.532
```

- **Speedup Assoluto:** la combinazione Single Kernel + Shared Memory ha abbassato drasticamente l'intera curva. Ad esempio, per il dominio più grande (8192x8192 a 16x16), il tempo crolla da 2.746s a 2.382s, confermando la necessità di sfruttare la gerarchia delle cache hardware negli algoritmi Memory-Bound.
- **Mantenimento del Profilo Architeturale:** pur essendo molto più veloce, la forma della curva di scaling rimane inalterata.

Utilizzo della Memoria Constant

La matrice `d_T_const` è a sola lettura. Spostarla nella constant memory permetterebbe di sfruttare una cache ultra-veloce dedicata.

La fattibilità è limitata, poiché la constant memory è molto piccola, non può contenere matrici così grandi. L'approccio funzionerebbe solo se in memoria costante salvassimo solo le coordinate delle sorgenti di calore, non un'intera maschera 2D della griglia.

MPI+CUDA

Questa sezione analizza la distribuzione del dominio su due GPU, in cui ciascun rank gestisce metà matrice e comunica le righe di bordo (*ghost cells*) via MPI.

Setup ed Esecuzione

Sono stati testati due scenari per valutare l'overhead di rete:

1. **Intra-nodo (1x2.slurm):** 2 GPU sullo stesso nodo fisico (wn41). La comunicazione MPI avviene in memoria condivisa (SHM).
2. **Inter-nodo (2x1.slurm):** 2 GPU su due nodi fisici diversi (wn41 e wn42). I messaggi viaggiano sull'infrastruttura di rete.

Analisi dei Tempi e Visualizzazione

```
# Intra-nodo
Rank 0: MPI: 0.019015 mS, copysend: 0.002077 mS, global: 0.226082 mS
# Inter-nodo
Rank 0: MPI: 0.005452 mS, copysend: 0.002182 mS, global: 0.063985 mS
```

- `copysend`: l'estrazione della riga 1D in VRAM è istantanea (~0.002 ms).

- **MPI** : rappresenta la latenza reale dell'ibrido e il vero collo di bottiglia (viaggio GPU->CPU via PCIe, rete MPI, e CPU->GPU via PCIe).

L'immagine `heatmap.png` , risultante dall'unione dei file generati dai due rank, dimostra che lo scambio bidirezionale delle *ghost cells* è avvenuto correttamente. L'assenza di discontinuità (strappi) sulla linea orizzontale centrale conferma la fluidità matematica dell'algoritmo distribuito.

